Large Language Models for Software Requirements Classification: Cross-Dataset Generalization and Cost-Efficiency

***Abstract*—Several recent studies report that prompted large language models (LLMs) match or exceed fine-tuned transformer baselines on software requirements classification, and some conclude that labelled training data is no longer needed. Those results are almost always obtained in-distribution, with training data or few-shot exemplars drawn from the corpus that also supplies the test items, and they rarely report what the technique costs to run. This paper re-examines the comparison after repairing both omissions. Fine-tuned encoders and prompted LLMs of three access tiers classify the same requirements from two public corpora, under the same tasks and the same scoring rule, at three levels of distribution shift: within one corpus, across held-out projects, and across an independently annotated corpus. Every encoder–LLM comparison is paired on identical items and corrected for multiplicity as one test family. The ranking of the two approaches is not stable across those levels: within a corpus the encoders are ahead and are never significantly beaten, but once the test corpus is annotated by someone else the ordering reverses. The reason is not that the transferred encoder fails to recognise the new vocabulary but that it carries the class balance of the corpus it was trained on—and the prompted models have the same weakness from another source, since their class balance is set by the wording of the instruction. Encoders remain an order of magnitude cheaper per item. The evaluation regime therefore belongs in the claim, because a result obtained in-distribution does not predict the ranking under a change of corpus and can reverse it.**

***Keywords*—requirements classification, large language models, fine-tuning, generalisation, distribution shift, empirical software engineering, cost-efficiency**

I. Introduction

Requirements classification assigns natural-language requirement statements to categories such as functional (FR) versus non-functional (NFR), or security-relevant versus not. ISO/IEC/IEEE 29148 defines requirements and their attributes across the life cycle [2], and classification is one of the first analytical steps after elicitation: getting it right supports early risk identification and architectural choice, and getting it wrong propagates downstream as rework [1]. Two decades of automation moved the task from weighted indicator terms [1], through supervised learning on engineered features [3], to fine-tuned transformers [4] and, since 2024, to prompted generative LLMs [6], [9]–[11], [14]; a systematic mapping documents the same progression [7].

It is worth being precise about what this work claims. No study we are aware of states that requirements classification is solved. What several report is parity or better between prompting and supervision on the corpora they use: Binkhonain and Alfayaz find that few-shot prompting matches or exceeds a fine-tuned transformer and conclude that prompting is viable where labelled data is scarce [9]; Tang et al. report an 8 B open model passing a fine-tuned baseline on PROMISE [11]; and Alhoshan et al. report that generative models perform competitively without task-specific training [10], extending earlier evidence that zero-shot embedding methods already made labelled data non-essential [5]. Together these make "prompting can replace fine-tuning" a reasonable reading of the state of the art, and it is that reading, rather than any individual paper's claim, that this study tests.

The reading rests on a measurement convention. Almost every published result is obtained in-distribution: the model is fine-tuned, or its exemplars selected, on the corpus that then supplies the test items. Where prior work reports improved generalisability it usually means transfer across *tasks* on the same corpora rather than to unseen projects or an independent dataset [9]—the property NoRBERT was motivated by, since it addressed classifiers over-fitting project-specific wording [4]. A second omission compounds the first: inference cost, latency and model size are seldom quantified, although they decide whether a technique is adoptable without a commercial API budget.

This paper asks what happens when both omissions are repaired. We hold the corpus, the items, the label sets and the scoring rule fixed, and vary only the distribution shift between what a model was fitted on and what it is scored on. We contribute a paired, multiplicity-controlled benchmark of twelve configurations over three tasks, two corpora and three regimes; evidence that the ranking depends on the regime; a diagnosis of the cause as a mis-set class prior that both families share; and an accuracy–cost comparison priced from the runs themselves. Three research questions organise the study. **RQ1**: how do open and commercial LLMs compare with fine-tuned encoders within one corpus? **RQ2**: how much is lost under cross-project and cross-dataset evaluation, and does the loss differ between the families? **RQ3**: what is the accuracy-versus-cost trade-off, and what is the cheapest freely available option that is good enough?

II. Related Work and the Evaluation Gap

A. From indicator terms to prompting

Rule-based work established the task and produced the PROMISE NFR collection [1]. Classical supervised learning followed, with SVMs separating FRs from NFRs [3]. Transfer learning then reset the state of the art: NoRBERT fine-tunes BERT and reports macro-F1 in the mid nineties on the PROMISE categories [4], a pattern the systematic mapping of Kaur and Kaur later confirmed [7]. Zero-shot embedding methods showed labelled data was not strictly necessary [5], and prompted generative LLMs now dominate.

B. Recent LLM studies (2024–2026)

El-Hajjami et al. [6] compared SVM and LSTM against GPT-3.5 and GPT-4 over five datasets and found no single best technique; Peer et al. [8] embedded LLM classification in a systems engineering workflow. Binkhonain and Alfayaz [9] report the study closest to ours: five LLMs under four prompting strategies over three tasks on PROMISE and SecReq, against a fine-tuned transformer that few-shot prompting matches or exceeds. Alhoshan et al. [10] ran more than 400 experiments over three open generative models, and Tang et al. [11] identified an over-prompting effect in which extra examples degrade accuracy. Chaudhary et al. [12] extended the task to app-store reviews, Vasudevan et al. [13] compared fine-tuned BERT and RoBERTa against GPT-4o, and Levy et al. [14] benchmarked three current models against expert judgment on PROMISE_exp.

C. What this literature does not measure

Table I summarises the evaluation coverage of these studies, and three patterns recur. *First*, evaluation is in-distribution: the same corpus supplies the training data or the exemplars and the test items. Only the app-review study [12] tests genuinely different text, and it reports markedly lower precision on several categories—a signal the literature has not followed up. *Second*, cost is not reported, even in the studies that use open models [10], [11]. *Third*, the same few small, ageing, public corpora underlie nearly every result, so contamination of LLM pre-training data is acknowledged but not addressed [27]. These gaps interact: if in-distribution parity is what motivates replacing supervision with prompting, and is also what corpus reuse and contamination would produce spuriously, then the comparison that settles the question has not been run.

TABLE I
Evaluation coverage of recent LLM-based requirements-classification studies. ✓ = reported; ✗ = not reported; ∼ = partially. "Cross-project" means held-out unseen projects; "cross-dataset" means an independently produced corpus.

Study	Open LLMs	Comm. LLMs	Cross-project	Cross-dataset	Cost
El-Hajjami et al. [6]	✗	✓	✗	✗	✗
Peer et al. [8]	✗	✓	✗	✗	∼
Binkhonain & Alfayaz [9]	✓	✓	✗	✗	✗
Alhoshan et al. [10]	✓	✗	✗	✗	✗
Tang et al. [11]	✓	✓	✗	✗	✗
Chaudhary et al. [12]	✗	✓	✗	∼	✗
Vasudevan et al. [13]	✗	✓	✗	✗	✗
Levy et al. [14]	✓	✓	✗	✗	✗

III. Study Design

Twelve model configurations classify the same requirements, under the same tasks, against the same frozen splits, scored by one rule, so every comparison is paired on identical items (Fig. 1). The pipeline runs as seven stages, each writing a content-hashed artefact the next consumes; every number in Section IV is read back from those artefacts by the scripts that typeset the tables and figures.

A. Data preparation

PROMISE_exp [16], an expanded release of the PROMISE NFR collection, contributes 969 requirements from 47 projects, each labelled FR or NFR and, when NFR, with one of eleven quality sub-types. SecReq [15] contributes 510 sentences from three industrial security specifications, each labelled security-relevant or not.

Preparation runs once, in the first stage. Texts are normalised and label strings mapped onto one controlled vocabulary, so both corpora express the security concept identically. Exact duplicates are removed—67 rows, all within a corpus rather than across the two—leaving 1,412 requirements in 50 project groups (968 from PROMISE_exp, 444 from SecReq). The fourteen split families are generated once with seed 42 and stored, keyed by identifiers pinned to the SHA-256 hash of each source file; because grouped-fold membership is library-version dependent, that file is consumed by later stages and never regenerated.

Two properties matter throughout. The first is a difference in class balance: security requirements are 12.9% of PROMISE_exp but 39.9% of SecReq, a threefold difference in prevalence for a concept both corpora annotate independently, and the cross-dataset regime is built on that contrast. The second is that the security label reaches the two corpora by different routes. In PROMISE_exp it derives from the security sub-class of the NFR taxonomy, so every PROMISE_exp item labelled security is also NFR and none of the 444 FRs is; in SecReq it applies to any security-relevant sentence, with no FR/NFR annotation at all. The cross-dataset regime therefore carries a definitional shift alongside the distributional one.

B. Tasks, label sets, and why they are not staged

Binary FR/NFR uses the native PROMISE_exp labels over all 968 PROMISE_exp items. *Binary security* is native in SecReq and derived in PROMISE_exp, giving the same task on two independently produced corpora. *NFR sub-type* classification uses the eleven-class taxonomy of Cleland-Huang et al. [1], whose categories align with the ISO/IEC 25010 quality characteristics [21].

The three tasks are evaluated independently. There is no cascade: the security classifier is not shown the output of the FR/NFR classifier, and every item is presented to it directly,

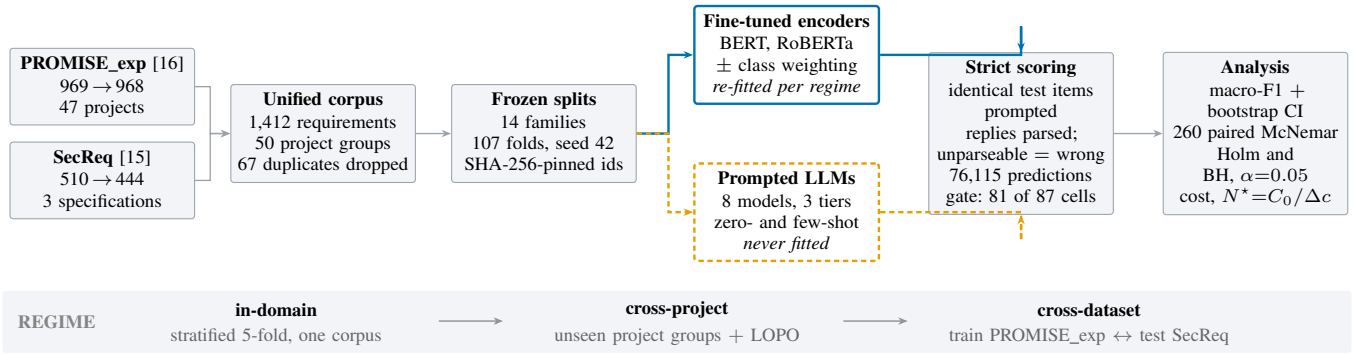

Fig. 1. Experimental architecture. One data path (left) feeds two model families that differ in exactly one respect: whether they consume training data. Both answer the *same* items, so every encoder–LLM comparison is paired item-for-item, and both pass through one strict scoring path (right). The band beneath is the study's independent variable: the distribution shift between what a model was fitted on and what it is scored on.

including the 444 PROMISE_exp items labelled FR. We say this because the PROMISE_exp *annotation* is hierarchical—security is a sub-class of NFR—so it would be easy to assume the *pipeline* is too. It is not, and deliberately so: a staged design would make the security score depend on the FR/NFR classifier's errors, and would have no counterpart in SecReq, where no FR/NFR labels exist. Keeping the tasks flat is what makes the security task comparable across corpora.

Several sub-type classes have single-digit support, so that task is evaluated at three granularities: eleven classes (524 items), the six most frequent (433) and the four most frequent (353). For an encoder the label set fixes the size of the classification head and the model is re-trained for each. For a prompted model there is no head to resize and no training at all, so it is fair to ask what top-4 and top-6 mean. They change three things. The candidate labels are enumerated inside the prompt itself, so at top-4 the model is offered four labels and at all-11 eleven; the evaluation frame changes, since an item enters the top-4 frame only if its true label is one of those four; and macro-F1 is computed over the full declared label set, so a model is penalised for every declared class it never predicts and that denominator grows with granularity. Granularity is a property of the question and of the scoring, not of the model weights, and is therefore measurable for both families.

C. Evaluation regimes

Fourteen frozen split families, comprising 107 folds, implement three regimes of increasing shift. **In-domain** is stratified 5-fold cross-validation within one corpus—the in-distribution setting prior work reports. **Cross-project** uses grouped folds holding out entire unseen project groups, plus a leave-one-project-out variant over the 47 PROMISE_exp projects. **Cross-dataset** fits on the security task of one corpus and tests on the other, in both directions: identical label sets, different annotation provenance.

The asymmetry between the two families is the analytical lever. Encoders are re-fitted at every regime on that regime's training folds; prompted models have no training distribution, so their scores cannot change with the regime, which is why the same score appears in every regime column of Table III. Every requirement is a test item exactly once within each encoder family. The prompted models answer stratified samples of those items—300 per binary cell and 200 per sub-type cell, in proportion to the class prior with a floor of ten per class—and the six local models additionally answer the full frames. Every comparison is paired on identical requirements, aligned by identifier rather than position.

D. The two model families, and why they are compared

Table II lists the twelve configurations. The comparison is between two architectures, and the difference is not incidental to the question. A fine-tuned encoder is a bidirectional, encoder-only transformer pre-trained with masked language modelling and then adapted by supervised training of a classification head over the pooled sentence representation; both its decision boundary and its class prior are learned from the labelled training folds. A prompted LLM is a decoder-only autoregressive model never adapted to the task: it receives the label vocabulary as text, answers in generated tokens, and its implicit class prior comes from pre-training and from the instruction rather than from any corpus of requirements. That contrast is what makes the comparison informative, since the two families should respond differently to distribution shift precisely by acquiring their prior from different places, and Section IV-C shows that they do. Prior work has argued that architecture affects results on this task [13], so we treat it as a declared property rather than a nuisance variable and report every result per configuration.

We fine-tune `bert-base-uncased` [18] and `roberta-base` [19] with a classification head (AdamW, lr 2×10^{-5}, batch 16, max length 128, linear decay; 3 epochs on the binary tasks, 10 on the sub-types), each with inverse-frequency class weighting, plus an unweighted control on the security task. The local tier runs six instruction-tuned models of 2.6–8.0 B parameters on a single free-tier NVIDIA T4 under 4-bit NF4 quantisation [20]; the hosted and commercial tiers access Llama-3.3-70B and Gemini 3.1 Flash-Lite through free API tiers. A ninth prompted model failed the reportability gate below and enters no table.

E. Prompt design and what it is meant to control

Each prompted model answers a fixed instruction naming the task, enumerating the label vocabulary and requiring exactly

TABLE II

THE TWELVE MODEL CONFIGURATIONS. "B" IS THE PUBLISHED PARAMETER COUNT OF THE EXACT CHECKPOINT, IN BILLIONS; GEMINI'S IS UNDISCLOSED AND IS RECORDED AS UNKNOWN RATHER THAN ESTIMATED.

Config.	Checkpoint / endpoint	B	Access
BERT (w / u)	`bert-base-uncased`	0.11	fine-tuned
RoBERTa (w / u)	`roberta-base`	0.13	fine-tuned
Gemma-2-2B	`gemma-2-2b-it`	2.61	local, 4-bit
SmolLM3-3B	`SmolLM3-3B`	3.08	local, 4-bit
Qwen2.5-3B	`Qwen2.5-3B-Instruct`	3.09	local, 4-bit
Phi-4-mini	`Phi-4-mini-instruct`	3.84	local, 4-bit
Qwen2.5-7B	`Qwen2.5-7B-Instruct`	7.62	local, 4-bit
Llama-3.1-8B	`Llama-3.1-8B-Instruct`	8.03	local, 4-bit
Llama-3.3-70B	`llama-3.3-70b-versatile`	70.6	hosted
Gemini 3.1 F-Lite	`gemini-3.1-flash-lite`	n/a	commercial

one word in reply, sent as a single user message. Decoding is greedy—temperature 0, at most twelve new tokens—so the only stochastic element is item sampling. The *base* FR/NFR instruction is reproduced verbatim below; the sub-type tasks use the same template with the permitted labels listed in place of "FR"/"NFR", which is how the top-4, top-6 and all-11 variants differ for a prompted model.

Fixing the instruction needs justifying, because a different wording could give a different result. *Where the wording comes from*: the template follows the zero-shot format used in the studies we compare against [9]–[11]—a task statement, a list of permitted labels, and an instruction to answer with the label alone. We did not tune it against the test items. *We do not claim it is the best prompt*; almost certainly it is not. What the design requires is a *fixed* instruction, so that the only quantity varying across the three regimes is the distribution shift; re-optimising it per regime would make any difference between regimes unattributable to the shift. The prompt is a control, not a recommendation. *What the wording is worth is measured, not assumed*: a sub-study re-runs two models over three phrasings of the same question—*base*, *terse* and *verbose*—with the answer contract held identical, over nine paired cells. An oracle picking the best wording per cell would gain a median of $+0.003$ macro-F1 and at most $+0.036$, far less than the differences this paper's conclusions rest on; and picking it would need labelled data from the target corpus, the resource prompting is meant to do without. A second sub-study is a few-shot ladder ($k \in \{2, 4, 8\}$ on the binary tasks, $k \in \{1, 2\}$ per class on the sub-types) with deterministic, class-balanced exemplars carved out *before* the evaluation frames are formed, so an exemplar is never a test item. The hosted and commercial tiers run zero-shot.

```
You are classifying a software requirement.
Decide whether it is a FUNCTIONAL requirement (FR) --
something the system must DO -- or a NON-FUNCTIONAL
requirement (NFR) -- a quality, constraint or property
such as performance, security or usability.
Answer with exactly one word: "FR" or "NFR".
few-shot only: k exemplars here
Requirement:
"""the requirement under test"""
Answer:
```

F. Parsing, metrics, and how to read them

Outputs are parsed by a strict, word-boundary-aware rule: a leading label wins; otherwise exactly one label named anywhere in the reply wins; a reply naming both labels or neither is unparseable and scored as wrong. This is harsher than the lenient protocols common in prior work and matches deployment, where an unusable answer is not a free pass. The rule runs at generation time and again in an idempotent repair pass over the archived outputs, so stored and scored predictions cannot diverge.

Four kinds of quantity are reported, and because they are easy to confuse we state what each is. *Macro-F1* is the unweighted mean of the per-class F1 scores over the full declared label set; it is primary because it does not let a model score well by ignoring a rare class, and the majority-class baseline in the last row of Table III gives every score a floor. *Relative loss* ($\Delta\%$) is the share of a configuration's own in-domain macro-F1 lost in a shifted regime, on the same items, and is defined only for the encoders, since only they have an in-domain fit to lose. *Predicted class share* is the fraction of test items a model assigns to a class—not an accuracy, but what the model takes the class balance to be, read against the true prevalence in the same items. It and relative loss are both percentages, so Section IV-B names the class and the corpus each time one is used. *Test counts* are numbers of statistical comparisons, not of items: each unit is one exact McNemar test between one encoder and one prompted model in one cell.

Uncertainty uses a stratified bootstrap: 2,000 resamples drawn within each true class (seed 42). Encoder–LLM comparisons use the exact two-sided McNemar test on paired disagreements [22]; all 260 are corrected as one family under Holm [23] and Benjamini–Hochberg [24] at $\alpha = 0.05$, with verdicts keyed on the BH-adjusted p (175 significant before correction, 170 after BH, 131 after Holm). A cell—one model, task, dataset and regime—is reported only if every class has at least 5 test items, the cell holds at least 40 items, and the model parsed at least 50% of its responses; 6 of 87 cells fail this gate and are excluded rather than imputed. The store holds 76,115 predictions.

G. Cost model

Every prediction row records token counts, wall-clock latency and, for the encoders, training time, so cost is computed from recorded quantities rather than estimated afterwards. The basis differs by tier, because the two kinds of model are billed differently.

Models that run on our own hardware are priced on compute. That covers the encoders and all six local open-weight models: they are free to download and carry no licence fee, so the only thing to charge for is the machine time they consume. We price it as an imputed rental of the benchmark GPU—an AWS `g4dn.xlarge` with one NVIDIA T4, at the published on-demand list rate for `us-east-1` of $0.5260 per hour, recorded in the manifest with the date it was checked. That is a list price for hardware, not for the model, and it assumes unbatched inference; the stored sensitivity table reports the same costs at the spot rate ($0.3162/h) and at batch sizes 8 and 32, which cut the local figures by roughly an order of magnitude. *Models behind an API are priced on tokens*, at the provider's published

list rate per million input and output tokens—$0.25/$1.50 for Gemini 3.1 Flash-Lite and $0.59/$0.79 for Llama-3.3-70B, taken from the providers' public rate cards in August 2026 and frozen into every prediction row at generation time, so a later price change cannot alter a stored result. Both were run on free tiers; the list rate is what the same volume would have cost.

Neither basis is a fixed property of the technique, and cost should be expected to move with model size and capability. Within the compute-priced tier it does: over the ten locally executed configurations, cost per item rises with parameter count (Spearman $\rho = 0.85$), so Section IV-D states the comparison as a ratio between tiers rather than an absolute price that would date. The ordering is not perfectly monotonic in size, because a locally run model is charged for the wall-clock time it occupies the GPU, and one that emits longer replies pays for them at equal size. Encoder cost separates the one-off training cost C_0 from the marginal per-item cost c_{enc}, giving a break-even volume against any prompted alternative

$$N^{\star} = \frac{C_0}{c_{\text{alt}} - c_{\text{enc}}}. \tag{1}$$

Because free-tier latency is dominated by provider-side queueing, wall-clock projections use per-request median latency.

IV. Results

Table III is the paper's single results table: all twelve configurations, the six task-by-corpus cells and every regime each supports, so encoders and prompted models are read off the same rows and columns. The encoders have one score per regime because they are re-fitted at each; a prompted model has one score per cell, written across the regime columns it spans, since a model that consumes no training data cannot score differently when that data changes.

A. RQ1: within one corpus, supervision is still ahead

Reading the ID columns of Table III, fine-tuned encoders hold the best score in five of the six cells: class-weighted BERT on FR/NFR and PROMISE_exp security, class-weighted RoBERTa on all three sub-type granularities. The exception is SecReq security, where 4-bit Qwen2.5-7B posts the better point estimate (0.846 against 0.836), a lead that does not reach significance (exact McNemar $p = 0.696$; BH-adjusted 0.774).

Three observations qualify the ranking. Scale does not order the results: the 2.6 B Gemma-2-2B beats the hosted 70 B model on PROMISE_exp security. The encoder advantage widens as the label set grows—from $+0.014$ on top-4 to $+0.095$ on all-11 over the best prompted model—as expected if supervision mainly buys the long tail of rare classes. Cell sizes differ, so the top-4 ranking was recomputed on the 59 items every model answered; the best encoder still leads there by $+0.036$, so the ordering is not an artefact of coverage. Over the 106 in-domain comparisons the encoder wins 67, 39 are inconclusive and none is lost (Fig. 2 (b)).

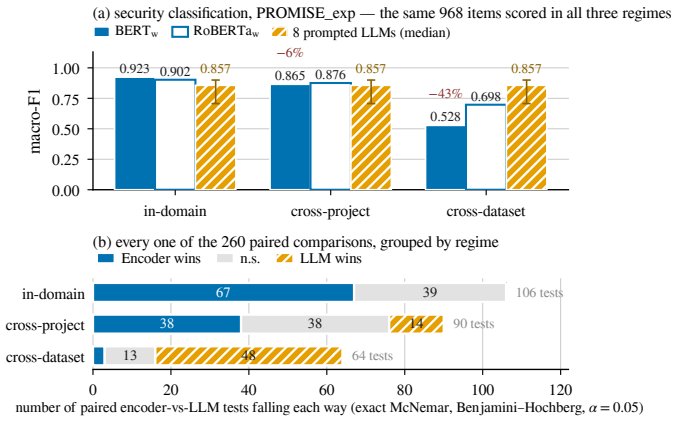

Fig. 2. (a) Security classification on PROMISE_exp; all three bars are scored on the same 968 items at every regime. The encoders are re-fitted at each regime; the third bar is the median of the eight prompted models, with the whisker spanning their range, and it is flat because those models are never fitted. The red annotations are BERT's relative loss against its own in-domain score. (b) The 260 paired encoder-versus-LLM comparisons, grouped by the regime the encoder was evaluated under. Each unit on the axis is one exact McNemar test, not one requirement.

B. RQ2: the ranking reverses under distribution shift

Cross-project transfer is survivable. Holding out unseen projects costs the encoders 3–12% of their in-domain macro-F1 on PROMISE_exp, and up to 30% on the much smaller SecReq, whose three project groups make a held-out group a third of the corpus. They remain ahead or tied in 76 of the 90 paired tests—the degradation NoRBERT anticipated [4]: real, but moderate.

Cross-dataset transfer reverses the ranking. The class-weighted BERT that scores 0.923 on PROMISE_exp security falls to 0.528 on the same items once its training corpus is swapped for SecReq: a loss of 42.8% of its in-domain macro-F1, the largest in the study. It belongs to the cross-dataset regime and should not be read against the cross-project losses above, whose maximum is 29.6%. RoBERTa is more robust (-23%), the unweighted controls behave like their weighted counterparts (-41% and -22%), and the reverse direction is milder still (-16 to -17%), while the prompted models, having nothing to re-fit, move only with the difficulty of the corpus. Over the 64 cross-dataset comparisons the prompted models win 48 and the encoders 3: the in-domain tally of 67–39–0 becomes 3–13–48, on the same twelve models and the same scorer.

Corpus-level averages hide per-project failure. On the leave-one-project-out variant of FR/NFR, over the 37 projects with comparable coverage, every configuration—encoder and prompted alike—scores 0.000 on at least one project and 1.000 on another. Only the middle separates them: the encoders' median per-project accuracy is 0.917 (s.d. 0.198) against medians of 0.545–0.750 (s.d. 0.213–0.301) for the small open models.

The mechanism is a shift in class balance. A drop in score does not say why a model failed, and the two candidate explanations—unfamiliar vocabulary and a shifted class prior—imply different remedies. The per-class evidence separates them

TABLE III

MACRO-F1 FOR ALL TWELVE CONFIGURATIONS, ACROSS EVERY TASK, CORPUS AND REGIME. ID = IN-DOMAIN, XP = CROSS-PROJECT, XD = CROSS-DATASET. BEST PER COLUMN IN BOLD; "—" MARKS A CELL BELOW THE REPORTABILITY GATE OF SECTION III-F, OR A REGIME THAT DOES NOT APPLY. A PROMPTED MODEL'S SCORE IS WRITTEN ONCE ACROSS THE REGIMES OF ITS CELL, BECAUSE IT CONSUMES NO TRAINING DATA AND THE REGIME CANNOT MOVE IT. ENCODER CELLS ARE CROSS-VALIDATED OVER ALL ITEMS (n = 968 / 444 / 353 / 433 / 524 BY CELL); PROMPTED CELLS ARE STRATIFIED SAMPLES OF THE SAME ITEMS (n = 59–960), AND ALL COMPARISONS ARE PAIRED ITEM-FOR-ITEM.

	Security						**FR/NFR**		**NFR sub-types** (PROMISE_exp)					
	PROMISE_exp			SecReq			PROMISE_exp		top-4		top-6		all-11	
Configuration	ID	XP	XD	ID	XP	XD	ID	XP	ID	XP	ID	XP	ID	XP
BERT_{w}	**0.923**	0.865	0.528	0.836	0.588	0.690	**0.908**	0.843	0.878	0.800	0.827	0.735	0.657	0.579
BERT_{u}	0.907	–	0.534	–	–	0.702	–	–	–	–	–	–	–	–
RoBERTa_{w}	0.902	0.876	0.698	0.824	0.668	0.696	0.896	0.853	**0.936**	0.835	**0.837**	0.753	**0.724**	**0.688**
RoBERTa_{u}	0.903	–	0.707	–	–	0.678	–	–	–	–	–	–	–	–
Gemini 3.1 Flash-Lite		0.829			0.713			0.867		0.922		–		–
Llama-3.3-70B		0.891			0.814			0.852		0.907		0.782		–
Qwen2.5-7B		0.884			**0.846**			0.656		0.876		0.824		0.576
Llama-3.1-8B		0.872			0.845			0.682		0.700		0.717		0.629
Gemma-2-2B		0.900			0.821			0.663		0.760		0.603		0.527
Phi-4-mini		0.798			0.635			0.727		0.727		0.712		0.599
SmolLM3-3B		0.841			0.717			0.562		0.763		0.693		0.482
Qwen2.5-3B		0.707			0.741			0.554		0.732		0.689		0.543
majority-class baseline		0.466			0.376			0.351		0.131		0.075		0.035

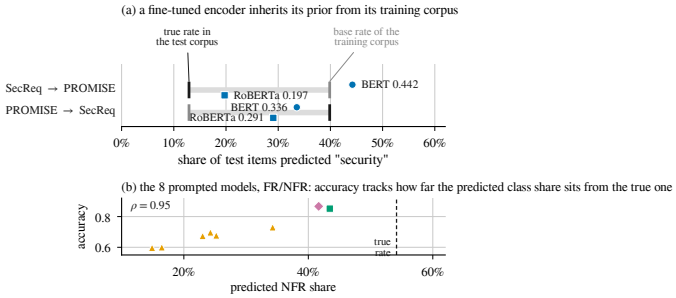

Fig. 3. One failure mode, two sources. (a) After cross-dataset transfer an encoder labels items at a rate pulled towards the base rate of the corpus it was *trained* on; the axis is the share of test items predicted *security*, not a score. (b) The prompted models show the same defect from another source.

(Fig. 3). After transfer from SecReq, BERT labels 44.2% of the PROMISE_exp items *security*, against a true prevalence of 12.9% in those items and 39.9% in the corpus it was trained on. That 44.2% is a predicted class share, not a loss of score, and its closeness to the 42.8% above is a coincidence. Its consequence is visible per class: precision on the security class falls from 0.847 to 0.210 while recall falls far less (0.888 to 0.720). The model still finds the security requirements; it cannot stop finding them—the signature of a shifted prior, not of a failure to read the text.

The picture is not uniform. RoBERTa carries less of its source prior across (19.7% predicted against 12.9% true) and loses correspondingly less, and in the reverse direction both encoders predict *below* the target prevalence (33.6% and 29.1% against 39.9%), so content differences contribute there too. The dominant term in the largest observed collapse is nevertheless a prior the model brought with it—an encouraging diagnosis, since prior shift is correctable without labels from the target corpus [25].

C. One failure mode, two sources

Turning the same instrument on the prompted models finds the same defect, arriving by a different route. On FR/NFR the true NFR share is 54.2%, and every prompted model predicts NFR less often than that, from 14.9% (Qwen2.5-3B) to 43.4% (Llama-3.3-70B). Their accuracies rank almost exactly with that miscalibration (Spearman $\rho = 0.95$, Fig. 3 (b)), so the small open models' weak FR/NFR scores reflect a systematic bias towards answering FR rather than an inability to read the requirement.

The wording of the question moves that balance directly, which makes the link causal rather than correlational. Across the nine paired cells of the phrasing sub-study the median spread between the three wordings is 0.044 macro-F1, but the largest is 0.485, for Gemma-2-2B on SecReq security. The predictions show what the *terse* wording does: it is the only variant phrased as a leading yes/no question, and under it Gemma-2-2B answers *security* for 96.7% of SecReq items against a true rate of 39.7%. This is not a formatting artefact—the parser accepted 99.98% of the sub-study's 11,518 responses—so the answers were well-formed and simply wrong. Prompt sensitivity is documented [26]; what is new is that it lands on the same failure mode as the encoders' transfer loss.

Few-shot exemplars help where they carry information about the class balance. Across the 90 paired comparisons on the six local models the median gain is $+0.024$ macro-F1, but the effect splits by task: slightly negative on the binary tasks (mean -0.007 over 54 comparisons) and positive on the sub-types (mean $+0.062$ over 36). Where the labels are a familiar convention the model already has a prior and examples mostly perturb it—Tang et al.'s over-prompting effect [11]; where they are an unfamiliar eleven-class taxonomy, the exemplars are the only statement of what the classes mean. These are one defect with two sources: the task needs a class prior that cannot be read off the requirement text, and each family acquires one from somewhere it should not—the encoder from its training

corpus, the prompted model from its pre-training and from the wording of the question.

D. RQ3: the accuracy–cost trade-off

Encoder inference costs \$0.0024 per 1,000 classifications on the imputed T4 rental of Section III-G. The prompted configurations range from \$0.0248 to \$0.0704 per 1,000 on the same basis, taking each model's median across its cells: 10 to 29 times the encoder, about 19 times at the medians. One-off fine-tuning is small—a median of \$0.0093 per deployable model, about 277 GPU-seconds over all folds of a family—so the break-even of Eq. (1) arrives quickly: over all 276 pairings $N^\star$ lies between 46 and 552 items, median 215. The Pareto frontier over reportable cells holds ten cells, nine of them encoders; the one prompted frontier point is 4-bit Qwen2.5-7B on SecReq security, which buys $+0.011$ macro-F1 for roughly $19\times$ the cost per item and is the strongest freely available option on the one cell where transfer is what is measured.

Two qualifications keep these figures in proportion: they are ratios measured on one machine at one date, and the model prices computation only—the annotation an encoder needs is excluded, which favours the encoder. Latency behaves differently from cost: the median per-request latency is 0.016 s for the encoders, 0.095 s for the hosted 70 B model, 0.194–0.356 s for the local open models and 0.614 s for the commercial API, while free-tier queueing inflates the hosted *means* by up to two orders of magnitude.

V. Discussion

Generalizability. The most consequential result is not that one family is better, but that the answer to "which is better?" changes between two protocols the literature treats as interchangeable, on the same corpora, items and scorer. A benchmark reporting only in-distribution numbers is not a weaker version of one that reports transfer; on this evidence it can report the opposite ordering. Two things follow: the regime belongs in the claim, next to the metric; and "generalisation" should mean held-out projects or an independently annotated corpus, not transfer across tasks within one dataset [9]. The limits of our own claim are equally explicit. We observe the reversal on one corpus pair, one task and one language, so the study establishes that it is possible and large when it happens, not that it occurs at any particular rate. The mechanism we identify is what would make it recur, and it is checkable on any new pair without re-running our models, by comparing the predicted class share against the target prevalence.

Consistency and reliability. Accuracy averaged over a corpus is not the only property a practitioner needs, and two kinds of instability appear, one in each family. The encoders are stable given a fixed training distribution and unstable when it moves: their scores change little across folds within a corpus but lose up to 42.8% of their in-domain macro-F1 when the training corpus is replaced. That failure is at least predictable from something checkable before deployment—the class balance of the corpus they were fitted on relative to the target. The prompted models are stable across regimes, by construction, but unstable with respect to the question: re-wording the same instruction, with the answer contract held fixed, moved macro-F1 by up to 0.485 in one cell. Pinning the prompt gives repeatable answers, but the level at which they are repeatable was set by a wording choice no in-distribution study would surface. Both families also show wide per-project variance, so a corpus-level mean without a per-project spread overstates how reliable either is.

Accuracy–cost trade-off. For a team with a few hundred labelled requirements from the domain it will operate in, and any non-trivial volume, a fine-tuned encoder of about 110 M parameters is both the more accurate and much the cheaper option. Entering an unlabelled or shifting domain changes the answer: a mid-size quantised open model gives up in-domain accuracy, degrades more gracefully, and runs on one consumer-class GPU without an API contract. What the results argue against is the implicit default of much recent work—fine-tuning on one corpus and assuming the result transfers. The trade-off is a ratio, not a price, and the annotation the encoder route needs sits outside the model.

Contamination and availability. Both corpora are old, small and public, so prompted in-domain scores may be inflated by pre-training contamination [27]. That bias would overstate the prompted models in-domain, which is where we report the encoders winning, so the in-domain advantage is conservative; and it cannot produce the cross-dataset reversal, since it would help them in both regimes equally. The pipeline, the splits, the prediction store and the scripts that regenerate every table and figure will be released with the paper, and scoring is a pure function of the stored outputs, so every number reproduces without re-running a model.

VI. Threats to Validity

Internal. The commercial tier ran at about 60 items per cell, where 23 of its 26 paired comparisons are inconclusive, so "never significantly beaten" partly reflects limited statistical power. Free-tier quotas caused missing-not-at-random dropout on some hosted configurations; the gate excludes the affected cells rather than imputing them. Local models ran 4-bit quantised, which may understate full-precision accuracy, and each configuration used a single seed, so small encoder-to-encoder margins should not be over-read.

Construct. GPU costs are imputed from a published rental rate rather than billed, and token prices are 2026 list rates, so the ratios are more durable than the absolute values. $N^\star$ is a pure cost crossover computed regardless of the accuracy gap, and the cost model prices computation only, so the annotation an encoder needs is excluded—which favours the encoder. Strict scoring counts format non-compliance as error, matching deployment but differing from the lenient protocols of some prior work. The prompt is fixed by design; the phrasing sub-study bounds what a better wording would be worth on two models, not on all eight.

External. Both corpora are small, dated and public, and their security label sets differ: PROMISE_exp's derives from the security sub-class and is non-functional by construction,

whereas SecReq annotates any security-relevant sentence, so the cross-dataset regime carries definitional as well as distributional shift, on one corpus pair. Results cover English requirements and one NFR taxonomy. All paired tests are exact and multiplicity-corrected as one family, and the splits are stored once and never regenerated because grouped-fold membership is library-version dependent.

VII. Future Work

Two extensions follow from the design and address its main limits. **Fine-tuning the LLMs.** The prompted models are used as they ship, which makes them a fixed reference across the regimes but leaves the comparison one-sided: the encoders are adapted to the task and the LLMs are not. Parameter-efficient fine-tuning—LoRA adapters over the same 4-bit checkpoints [20]—would let an LLM be re-fitted at each regime exactly as the encoders are, and would separate two explanations of the cross-dataset result that the present design cannot: whether prompted models transfer better because they are decoder-only generative models, or simply because they were never fitted to a source corpus. Section IV-C predicts the latter—a fine-tuned LLM should acquire the same source-corpus prior and lose the transfer advantage with it. That is testable on the pipeline as it stands, and the cost model would then cover training as well as inference for both families.

A third corpus. The reversal is observed on one corpus pair, so the next step is a third independently annotated source; PURE [17] is the natural candidate, being a public collection of complete requirements documents rather than isolated sentences. It would turn one cross-dataset contrast into several and let the prevalence gap be varied rather than merely observed, showing whether the size of the loss tracks that gap, as the prior-shift explanation predicts. It would also weaken the contamination threat, since a less-reused corpus is less likely to sit in a model's pre-training data. Prior adjustment [25] could then be evaluated as a remedy rather than left, as here, as a diagnosis.

VIII. Conclusion

We compared four fine-tuned encoder configurations against eight prompted LLMs on three requirements-classification tasks, holding the items, the label sets and the scoring rule fixed and varying only the distribution shift between what a model was fitted on and what it was scored on. Within a corpus the encoders are ahead and are never significantly beaten; across an independently annotated corpus the ordering reverses, while the encoders remain an order of magnitude cheaper per item. The cause is the same in both families: a class prior acquired from somewhere other than the corpus being classified. An in-distribution score is therefore not a partial answer to which approach to adopt; it can be the wrong one.

References

[1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, 2007.

[2] *ISO/IEC/IEEE 29148:2018(E) — Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, pp. 1–104, 30 Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.

[3] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 490–495.

[4] T. Hey, J. Keim, A. Koziolek, and W. F. Tichy, "NoRBERT: Transfer learning for requirements classification," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, 2020, pp. 169–179.

[5] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Inf. Softw. Technol.*, vol. 159, art. 107202, 2023.

[6] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT," arXiv:2311.11547, 2024.

[7] K. Kaur and P. Kaur, "The application of AI techniques in requirements classification: A systematic mapping," *Artif. Intell. Rev.*, vol. 57, art. 57, 2024, doi: 10.1007/s10462-023-10667-1.

[8] J. Peer, Y. Mordecai, and Y. Reich, "NLP4ReF: Requirements classification and forecasting—From model-based design to large language models," in *Proc. IEEE Aerospace Conf.*, 2024, pp. 1–16.

[9] M. Binkhonain and R. Alfayaz, "Are prompts all you need? Evaluating prompt-based large language models for software requirements classification," *Requirements Eng.*, 2025.

[10] W. Alhoshan, A. Ferrari, and L. Zhao, "How effective are generative large language models in performing requirements classification?" arXiv:2504.16768, 2025.

[11] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, "The few-shot dilemma: Over-prompting large language models," arXiv:2509.13196, 2025.

[12] M. Chaudhary, C. Jain, and P. R. Anish, "Exploring zero-shot app review classification with ChatGPT: Challenges and potential," arXiv:2505.04759, 2025.

[13] P. Vasudevan, S. Reddivari, S. Ahuja, N. Niu, S. Kumar, B. Jung, and E. Gamess, "Requirements classification using fine-tuned transformer models: A comparative study of BERT, RoBERTa, and GPT-4o," in *Proc. ACM Southeast Conf. (ACMSE)*, 2026, pp. 199–204.

[14] O. Levy, I. Dikman, N. Levy, and M. Winokur, "AI-assisted requirements engineering: An empirical evaluation relative to expert judgment," arXiv:2604.15222, 2026.

[15] E. Knauss, S. Houmb, K. Schneider, S. Islam, and J. Jürjens, "Supporting requirements engineers in recognising security issues," in *Proc. REFSQ*, 2011, pp. 4–18.

[16] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, "Software engineering repositories: Expanding the PROMISE database," in *Proc. XXXIII Brazilian Symp. Softw. Eng. (SBES)*, 2019, pp. 427–436.

[17] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.

[18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[19] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[20] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10088–10115.

[21] *ISO/IEC 25010:2011, Systems and Software Engineering—SQuaRE—System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[22] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[23] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[24] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.

[25] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002.

[26] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. ICML*, 2021, pp. 12697–12706.

[27] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023, pp. 10776–10787.